Tiny-GEMM: Packed INT4 Triton GEMM for Decode-Heavy LLM Inference

Robert Zhang[1]

[1]Purdue University
[1]zhan4808@purdue.edu

Abstract

Small-batch LLM decoding is dominated by narrow GEMMs that stress memory bandwidth and launch overhead rather than peak FLOPs. We present Tiny-GEMM, a packed INT4 GEMM kernel in Triton targeted at decode-heavy shapes, and a measurement-driven analysis of when quantization helps or hurts. We compare FP16, dequantized FP16, and INT4 fused kernels, attribute performance using hardware counters, and isolate dequantization overhead with microbenchmarks. Results on an A10G show up to 3.7× speedup for wide FFN decode shapes, while narrow projections regress when dequantization dominates. We provide a regime model and decision rule that predicts when INT4 helps, backed by hardware utilization and roofline evidence. Our evaluation connects kernel behavior to systems-level inference constraints.

1 Introduction

LLM inference workloads differ sharply from training: decoding operates at batch sizes of 1–8 with skinny matrices and tight latency budgets. These shapes are often memory-bound, and naive quantization can underperform if dequantization overhead is not amortized. This work explores packed INT4 GEMM in Triton and provides a systems-oriented evaluation emphasizing utilization, bottleneck regimes, and deployment-relevant shapes.

We argue that decode inference occupies a distinct optimization regime: launch overhead and memory traffic dominate, so quantization benefits depend on amortizing dequantization cost rather than peak compute.

Across representative decode shapes, Tiny-GEMM achieves up to 3.7× speedup over FP16 on wide FFN projections (M=1, K=4096, N=14336).

Contributions.

- A decode-focused INT4 GEMM kernel with static configuration selection and measurement-driven tuning.
- A regime analysis of INT4 decode GEMMs that identifies when quantization helps or hurts, supported by FP16 and dequantized FP16 baselines.
- A causal attribution methodology combining hardware counters and microbenchmarks to explain performance transitions.

2 Background and Motivation

Decode GEMMs (e.g., Q/K/V projections and FFN layers) typically use $M \in \{1, 2, 4, 8\}$ and hidden dimensions in the 4K–16K range. For these shapes, the dominant costs are memory movement and kernel launch overhead, not peak compute. This motivates INT4 weight packing and fused computation to reduce traffic and improve utilization, while requiring careful attention to dequantization and launch overheads in the small-batch regime.

3 Method

We implement packed INT4 GEMM in Triton with per-tensor quantization and bit-packed weight tensors. The kernel unpacks INT4 values into FP32 accumulators inside the kernel and uses static configurations keyed by shape family and batch bucket. We evaluate three deployment baselines:

- **FP16**: `torch.matmul` as the standard baseline.
- **Dequantized FP16**: INT4 quantize → dequant → FP16 GEMM.
- **INT4 fused**: packed INT4 GEMM kernel (Tiny-GEMM).

The current kernel performs dot products in FP32 and does not yet leverage INT4 tensor core MMA instructions; exploiting INT4 MMA is future work.

Figure 1: Tiny-GEMM kernel flow: packed INT4 weights are unpacked in shared memory, accumulated in FP32, and written to output.

4 Experimental Setup

All experiments run on an NVIDIA A10G GPU. We focus on decode shapes derived from Llama-style models: Q/FFN projections with K, N in $\{4096, 14336\}$ and KV projections with $N = 1024$. Profiling uses Nsight Compute; wall-clock timings are reported from the benchmark harness (profilers replay kernels and inflate timings).

Each reported latency is the median of 50 runs after 10 warmup iterations; the coefficient of variation is below 9% across repetitions in our sweep. All FP16 baselines use vendor-optimized libraries (`torch.matmul`/cuBLAS) under identical tensor shapes and launch conditions. Both FP16 and INT4 accumulate into FP32, and timings synchronize before and after loops to exclude host-side overhead.

Table 1: Representative decode shapes (Llama-style).

Layer	M	K	N
Q/K/V proj	1–8	4096	4096
KV proj	1–8	4096	1024
FFN up	1–8	4096	14336
FFN down	1–8	14336	4096

5 Results

5.1 Representative Shapes and Key Results

We focus on Llama-style decode GEMMs with small batch ($M \leq 8$) and large hidden dimensions ($K, N \in \{4096, 14336\}$), plus a narrow KV projection ($N = 1024$). Wall-clock timings are from our benchmark harness (profilers replay kernels and inflate timings); these shapes dominate autoregressive decode in Llama-style transformers.

Shape	FP16 (ms)	INT4 (ms)	Speedup	Bound
KV proj	0.027	0.043	0.62x	Dequant
Q proj	0.075	0.047	1.58x	Mixed
FFN up	0.239	0.067	3.58x	Memory
FFN down	0.258	0.152	1.69x	Memory

Table 2: Summary of key model shapes (M=1).

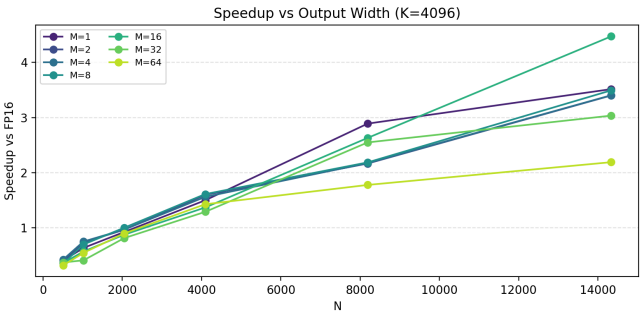
5.2 Where Does INT4 Help?

We first identify the empirical performance regimes across representative decode GEMMs. Figure 2a reports speedup relative to FP16 as output width N increases at fixed $K = 4096$, isolating the effect of layer dimension on performance.

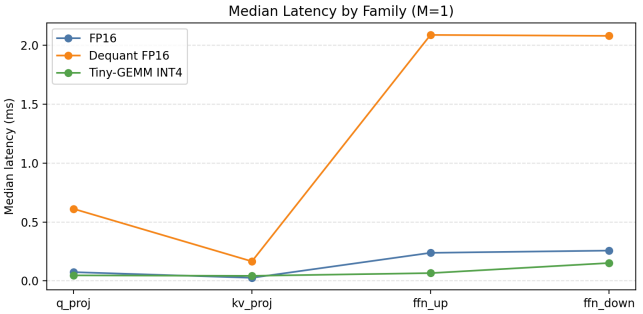
INT4 underperforms for narrow projections (e.g., KV projections with small N), but delivers substantial gains as matrix width increases, reaching up to 3.7× speedup for wide FFN layers. Figure 2b confirms this behavior at the layer-family level: projection layers show limited or negative benefit, while FFN layers consistently improve. The KV projection regression persists for both $M = 1$ and $M = 8$, indicating a geometry-driven transition rather than a batch-size effect. Across all $M \in \{1, \ldots, 64\}$, the transition remains near $N \approx$ 2–4K for $K = 4096$.

5.3 Prefill vs Decode

We compare prefill-like ($M \gg 1$) and decode-like ($M \leq 8$) regimes using the same layer-family grouping (Figure 3). Prefill increases effective arithmetic intensity by amortizing quantization overhead across more work, yielding consistent INT4 speedups across shapes.



(a) Speedup vs output width.



(b) Latency by layer family.

Figure 2: Decode performance across representative shapes.

Decode operates in a latency-dominated regime where small batch sizes expose fixed overheads such as dequantization and launch cost, making INT4 benefits shape-dependent. For example, FFN-up improves by roughly 3.5× in decode while KV projection remains below 1×; in prefill, FFN-up is about 3.0× and KV improves to roughly 2.3×.

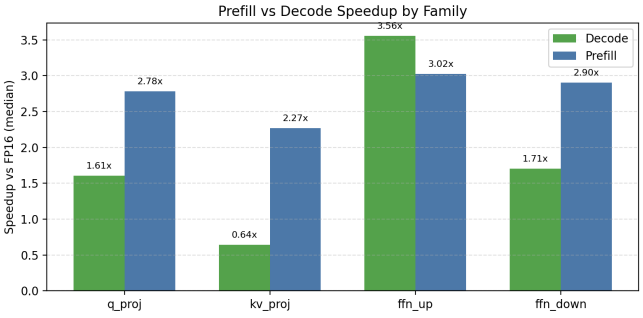


Figure 3: Prefill vs decode speedup by family.

5.4 Why Does INT4 Help (or Hurt)?

We attribute the observed performance regimes using hardware counters and a targeted dequantization microbenchmark. Together, these measurements explain why INT4 provides large gains for some decode shapes while regressing on others.

Figure 4 compares hardware utilization between FP16 and INT4 kernels. On the A10G, FP16 decode GEMMs operate close to the bandwidth limit: memory throughput reaches approximately 75–77% of peak DRAM bandwidth while

achieved compute throughput remains relatively low, indicating a strongly memory-bound regime where performance is limited by weight movement rather than arithmetic capability. INT4 weight packing reduces memory traffic and shifts the execution regime.

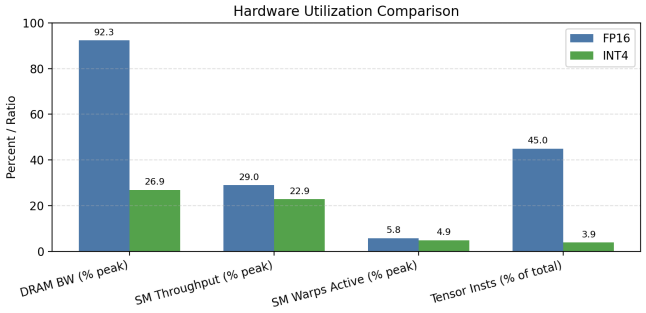


Figure 4: Hardware utilization comparison (FP16 vs INT4).

However, reduced memory traffic alone does not guarantee speedup. The dequantization microbenchmark in Figure 5 isolates the cost of unpacking INT4 weights during execution. For narrow projection layers (e.g., KV projections), total work per launch is small, and the fixed dequantization overhead constitutes a significant fraction of runtime. As a result, INT4 kernels can underperform FP16 despite lower memory traffic.

In contrast, wide FFN projections provide sufficient arithmetic work to amortize dequantization cost, allowing INT4 to achieve substantial latency improvements. Overall, INT4 improves decode performance when reduced memory movement outweighs added dequantization overhead.

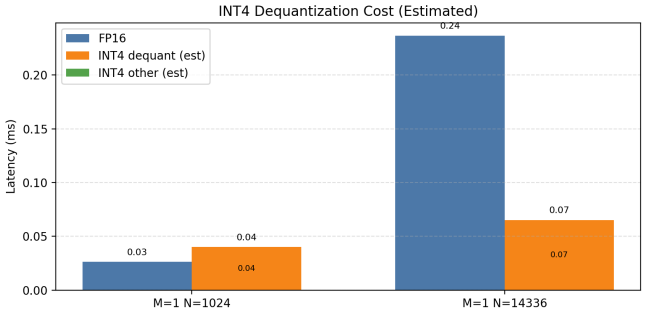


Figure 5: Dequantization breakdown for representative decode shapes.

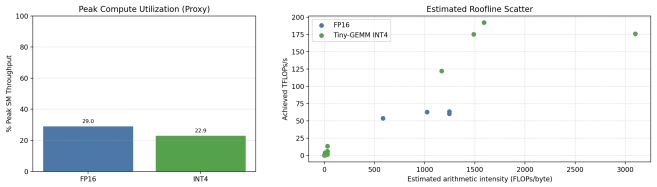
5.5 A Regime Model for INT4 Decode GEMMs

We summarize the observed performance behavior using a simple latency decomposition:

$$T_{\text{total}} = T_{\text{launch}} + T_{\text{mem}}(W) + T_{\text{dequant}} + T_{\text{compute}}. \quad (1)$$

This formulation captures the competing costs exposed in decode workloads. Narrow projections perform limited arithmetic per launch and are therefore dominated by fixed overheads, particularly T_{dequant} and launch latency. In contrast, wide FFN layers perform substantially more work per memory access, making execution primarily bandwidth-limited through T_{mem}.

INT4 weight packing reduces memory traffic, decreasing T_{mem} and shifting execution toward a mixed compute–memory regime. Compute utilization and roofline views support this model. INT4 achieves $\tilde{2}3\%$ peak SM throughput vs $\tilde{3}2\%$ for FP16 (about 28% lower), reflecting reduced compute pressure under the INT4 kernel. The roofline view in Figure 6b illustrates this shift away from bandwidth saturation.



(a) Peak compute utilization (SM throughput).

(b) Roofline view (arithmetic intensity vs TFLOPs).

Figure 6: Utilization regime view. INT4 shifts execution away from bandwidth saturation and toward a mixed compute–memory regime.

INT4 weight packing introduces a transition boundary in arithmetic-intensity space separating overhead-limited and bandwidth-limited regimes. In our sweep, the boundary occurs near $\alpha \approx 8$ FLOPs/byte.

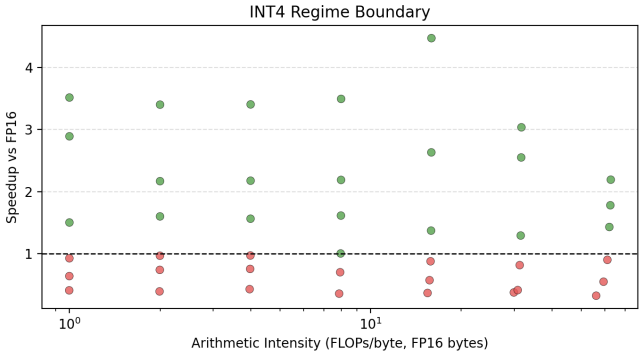


Figure 7: Regime boundary: speedup vs arithmetic intensity.

Figure 7 visualizes this transition. Shapes below the boundary remain dominated by fixed overheads and therefore see limited benefit from INT4, while shapes above it achieve consistent speedups as reduced memory movement outweighs dequantization cost.

This model consolidates the empirical observations from earlier sections: narrow projections operate below the boundary, while increasing N raises arithmetic intensity and shifts execution into a bandwidth-dominated regime where INT4 becomes advantageous.

INT4 Decision Rule. INT4 improves decode latency when the memory savings from FP16 → INT4 exceed dequantization cost: $T_{\text{mem}}^{\text{FP16}} - T_{\text{mem}}^{\text{INT4}} > T_{\text{dequant}}$.

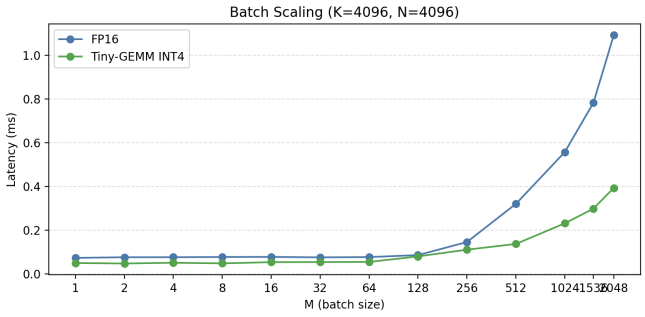
In practice, this boundary provides a deployment guideline: quantization is most effective for wide decode GEMMs and

FFN layers, while narrow projection layers may benefit more from kernel fusion or launch amortization strategies than from aggressive weight compression.

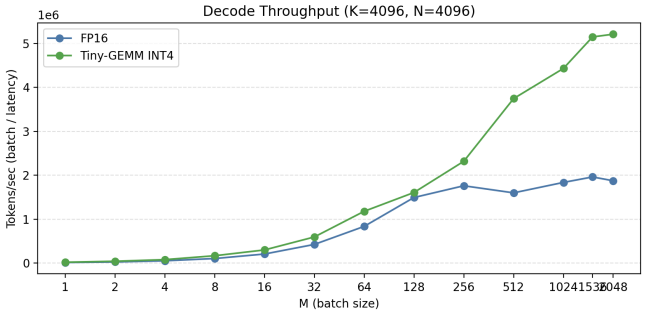
5.6 Systems-Level View

Because autoregressive decoding executes one token at a time, we connect kernel-level behavior to end-to-end decode constraints through batch scaling and throughput analysis. These latency–throughput tradeoffs directly determine user-visible response time in interactive LLM serving.

Figure 8 shows that latency grows sublinearly with batch size while throughput increases substantially, reflecting improved hardware utilization as launch overhead is amortized.



(a) Latency vs batch ($K = N = 4096$).



(b) Throughput (tokens/sec) vs batch ($K = N = 4096$).

Figure 8: Batch scaling tradeoffs for a representative decode shape.

For small batches ($M = 1$–4), decode latency is dominated by fixed kernel overheads, limiting the benefit of additional compute efficiency. As batch size increases, INT4 benefits become more consistent because memory traffic and compute utilization rather than launch overhead determine performance.

These results highlight an important systems tradeoff: optimizations that improve single-token latency do not always maximize throughput, and deployment targets (interactive latency vs. serving throughput) determine whether INT4 provides meaningful gains. This behavior mirrors production serving systems, where batching improves throughput but increases token latency, creating a practical optimization tradeoff.

5.7 Regime Map and Kernel Composition

We connect kernel-level behavior to system-level impact by examining where decode runtime is actually spent and how performance regimes distribute across model shapes.

Figure 9 shows that a small set of wide FFN GEMMs dominates total CUDA execution time during decode. Although transformer layers contain many projection operations, runtime is heavily concentrated in high-dimensional FFN computations. Consequently, optimizing these shapes yields disproportionate end-to-end latency improvements, motivating decode-focused specialization rather than uniform optimization across all layers.

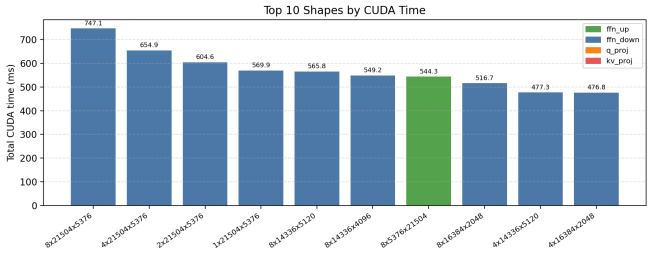


Figure 9: Top shapes by CUDA time (Nsight Systems).

Figure 10 complements this view by visualizing performance regimes across decode shapes. INT4 provides consistent speedups only once arithmetic intensity exceeds a threshold determined primarily by output width and hidden dimension. Regions with small N remain dominated by dequantization and launch overhead, explaining regressions observed for KV-like projections.

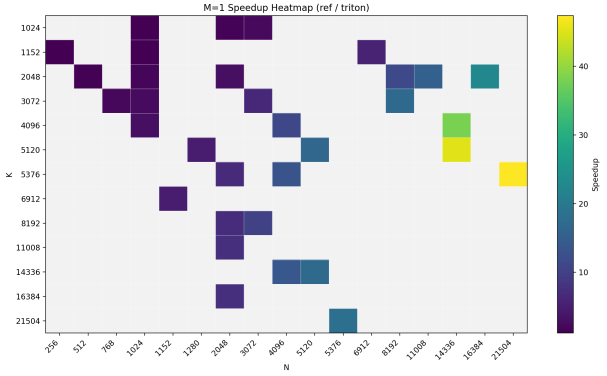


Figure 10: Decode regime map (M=1) across K and N.

Together, these results unify the regime analysis with runtime behavior: the layers that dominate decode execution are those above the regime boundary where INT4 provides the largest benefit.

6 Discussion

The results support a clear regime interpretation: INT4 helps when arithmetic intensity is sufficient to amortize dequantization and when memory traffic is a dominant bottleneck. For narrow projections (e.g., KV), fixed overheads and limited parallelism can outweigh the bandwidth savings. These findings align with practical inference constraints in decode-heavy

serving. Our results suggest that weight-only INT4 should not be applied universally in decode pipelines; kernel-aware deployment decisions are necessary to avoid regressions on narrow projections and to target the shapes that dominate runtime.

7 Related Work

Quantization for inference has been explored extensively in SmoothQuant, GPTQ, and AWQ, among others, focusing on accuracy-preserving weight-only and activation-aware techniques [7, 2, 3]. Systems efforts such as FlashAttention-2 and TensorRT-LLM emphasize kernel specialization and end-to-end inference throughput [1, 6]. CUTLASS and cuBLASLt provide highly optimized GEMM kernels that serve as deployment baselines [5, 4]. Tiny-GEMM complements these lines of work by focusing on the decode regime and providing a measurement-based model of when weight-only INT4 improves latency. In this sense, Tiny-GEMM bridges quantization research (accuracy and compression) with kernel-level inference optimization by isolating regime transitions under decode-specific constraints.

8 Limitations

This study targets a single GPU and a fixed set of decode shapes. The kernel is not yet optimized for all regimes (e.g., split-K for $M = 1$), and the INT4 kernel does not currently exploit tensor core INT4 MMA pipelines. Future work should extend to additional hardware (L4/A100), incorporate split-K or persistent kernel strategies, and explore INT8/FP8 comparisons. We also do not evaluate end-to-end model accuracy under quantization, which is orthogonal to the kernel performance focus of this work.

9 Conclusion

Tiny-GEMM demonstrates that packed INT4 kernels can significantly improve decode performance for wide FFN shapes while exposing the boundaries where dequant overhead dominates. By pairing multi-baseline benchmarks with hardware counters and targeted microbenchmarks, we provide a systems-level narrative of when and why INT4 helps in real inference settings.

References

[1] Tri Dao et al. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

[2] Elias Frantar et al. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[3] Yihong Lin et al. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[4] NVIDIA. cublaslt documentation. `https://docs.nvidia.com/cuda/cublas/index.html#cublasLt`, 2023.

[5] NVIDIA. Cutlass: Cuda templates for linear algebra subroutines. `https://github.com/NVIDIA/cutlass`, 2023.

[6] NVIDIA. Tensorrt-llm. `https://github.com/NVIDIA/TensorRT-LLM`, 2023.

[7] Wei Xiao et al. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.